Git 版本控制实验报告

姓名：苏丹

学号：24300240013

实验环境：VMware、Linux、zsh、Git、GitHub

个人仓库：https://github.com/24300240013-fudan-edu-cn/ICS

模板仓库：https://github.com/ICS-26Fall-FDU/GitLab

一、文档问题回答

1. 之前是否有多人协同开发经历？如何分工协作？

有。我曾参与多人协作的游戏开发，团队按照工作内容分工，我主要负责音效和音乐制作。我们通过 Discord 沟通需求、交流进度和反馈意见，配合完成开发任务。

2. Git 为什么设计“暂存—提交”两个步骤？

暂存区用于选择和整理本次准备提交的修改，提交则将暂存区中的内容保存为一个版本。

一次开发可能包含多个任务的修改。通过暂存区，可以按文件甚至按部分修改组织提交，避免把无关改动混在一起。同时，提交前可以检查暂存内容，使每次提交主题明确，便于后续查看历史、排查问题和回退版本。

3. git branch 和 git branch -a 的区别是什么？

不带其他参数时，git branch 列出本地分支，git branch -a 同时列出本地分支和远程跟踪分支，例如 remotes/origin/main。

远程跟踪分支反映本地保存的远程分支状态。git branch -a 不会自动联网刷新信息，如需更新，可先执行 git fetch。

参考资料：https://git-scm.com/docs/git-branch

4. 怎样修改 main.c 会出现合并冲突？

从两个分支的共同版本出发，在 main 和 feature 分支中，将 main.c 的同一行分别改为不同内容并提交。合并时，Git 无法自动确定最终应保留哪种内容，通常会产生冲突，需要人工处理。

二、实验步骤

1. 配置 Git 和 GitHub SSH

在 Linux 中执行以下命令，设置提交身份和默认分支名：

```
git config --global user.name "24300240013-fudan-edu-cn"

git config --global user.email "24300240013@m.fudan.edu.cn"

git config --global init.defaultBranch main
```

将已有的 ~/.ssh/id_ed25519.pub 公钥添加到 GitHub 的 SSH and GPG keys 页面，然后执行：

```
ssh -T git@github.com
```

终端返回 “You’ve successfully authenticated”，说明 SSH 认证成功。

2. 使用模板创建个人仓库

在课程模板仓库页面选择 Use this template → Create a new repository，创建个人仓库 ICS。

通过 SSH 克隆并进入仓库：

```
git clone git@github.com:24300240013-fudan-edu-cn/ICS.git
```

```
cd ~/ICS
```

3. 完成 main.c 的 TODO 并提交

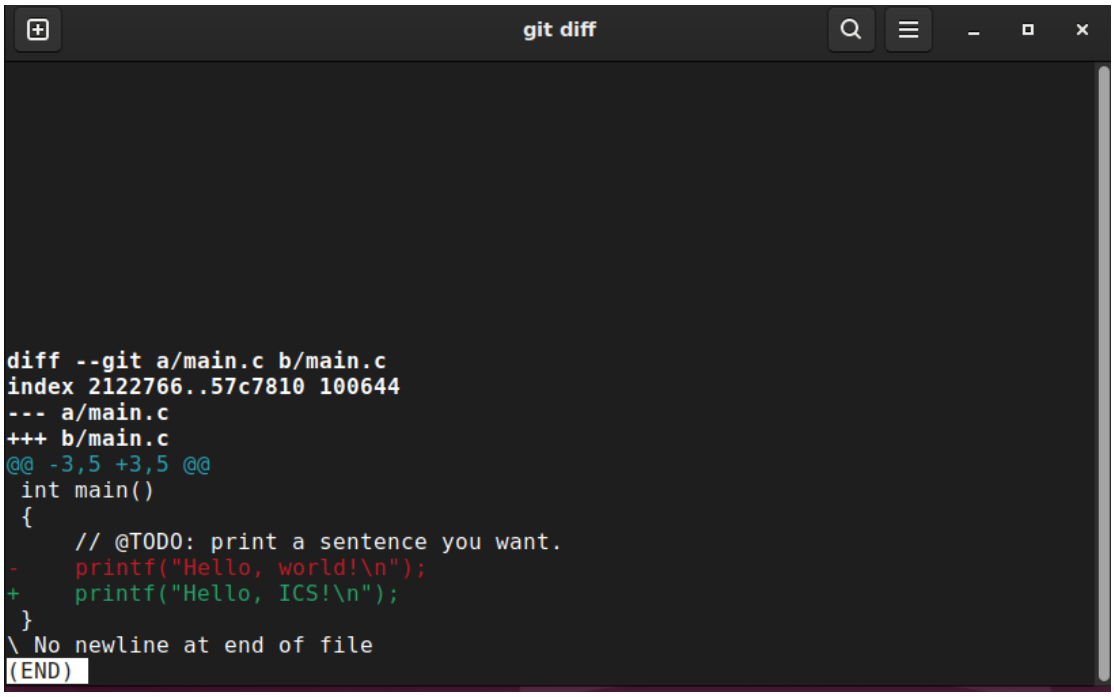
将原来的输出语句：

```
printf("Hello, world!\n");
```

修改为：

```
printf("Hello, ICS!\n");
```

使用 git diff 检查修改：



暂存并提交：

```
git add main.c
```

```
git commit -m "feat: complete main.c TODO"
```

提交编号为 b18af92。

4. 创建 feature 分支并提交修改

创建并切换到 feature 分支：

```
git switch -c feature
```

将输出语句修改为：

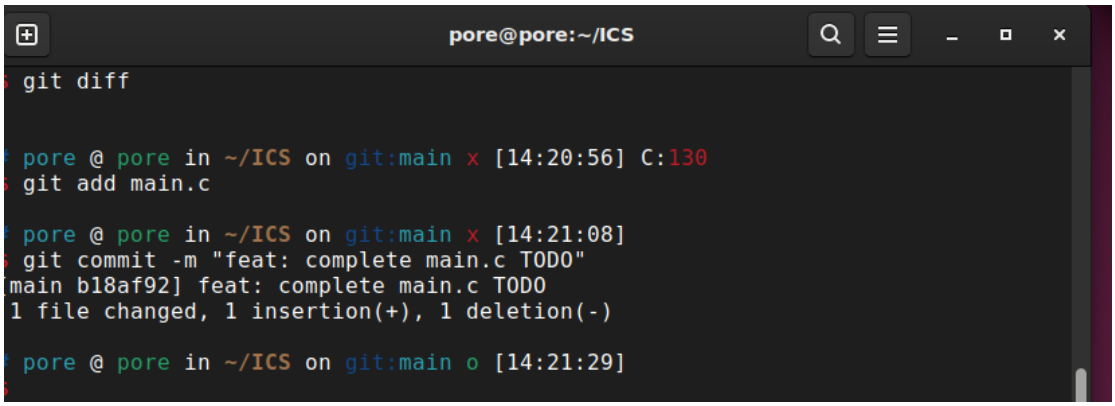
```
printf("Hello from feature!\n");
```

执行：

```
git add main.c
```

```
git commit -m "feat: update greeting on feature"
```

提交编号为 2819604。



5. 在 main 分支修改同一行并提交

切换回 main：

```
git switch main
```

将同一行输出语句修改为：

```
printf("Hello from main!\n");
```

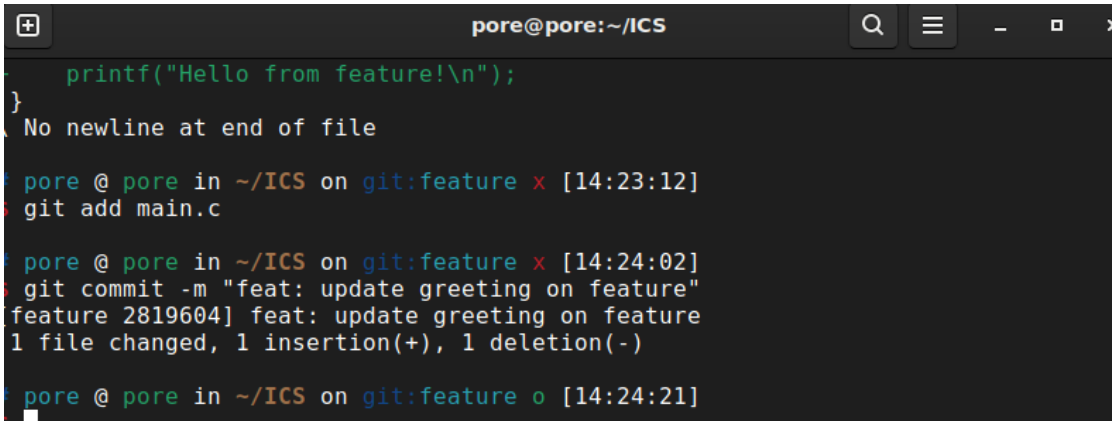
执行：

text 复制

```
git add main.c
```

```
git commit -m "feat: update greeting on main"
```

提交编号为 518710f。两个分支由共同版本出发，对同一行作出了不同修改。



6. 合并分支并处理冲突

在 main 分支执行：

```
git merge feature
```

Git 提示：

```
CONFLICT (content): Merge conflict in main.c
```

Automatic merge failed; fix conflicts and then commit the result.

```
# pore @ pore in ~/ICS on git:main o [14:27:51]
$ git merge feature
Auto-merging main.c
CONFLICT (content): Merge conflict in main.c
Automatic merge failed; fix conflicts and then commit the result.
```

查看 main.c，可以看到以下冲突块：

```
<<<<<<< HEAD
    printf("Hello from main!\n");
=======
    printf("Hello from feature!\n");
>>>>>>> feature
```

其中 HEAD 对应当前 main 分支的内容，另一部分对应 feature 分支的内容。

根据预期输出整合两边修改，删除冲突标记，最终 main.c 为：

```
#include <stdio.h>


int main()
```

```
{
    printf("Hello from main and feature!\n");
    return 0;
}
```

执行以下命令，标记冲突已解决并完成合并提交：

```
git add main.c
git commit -m "merge: resolve main.c conflict"
```

合并提交编号为 bf2b8d8。

7. 检查结果并推送

执行：

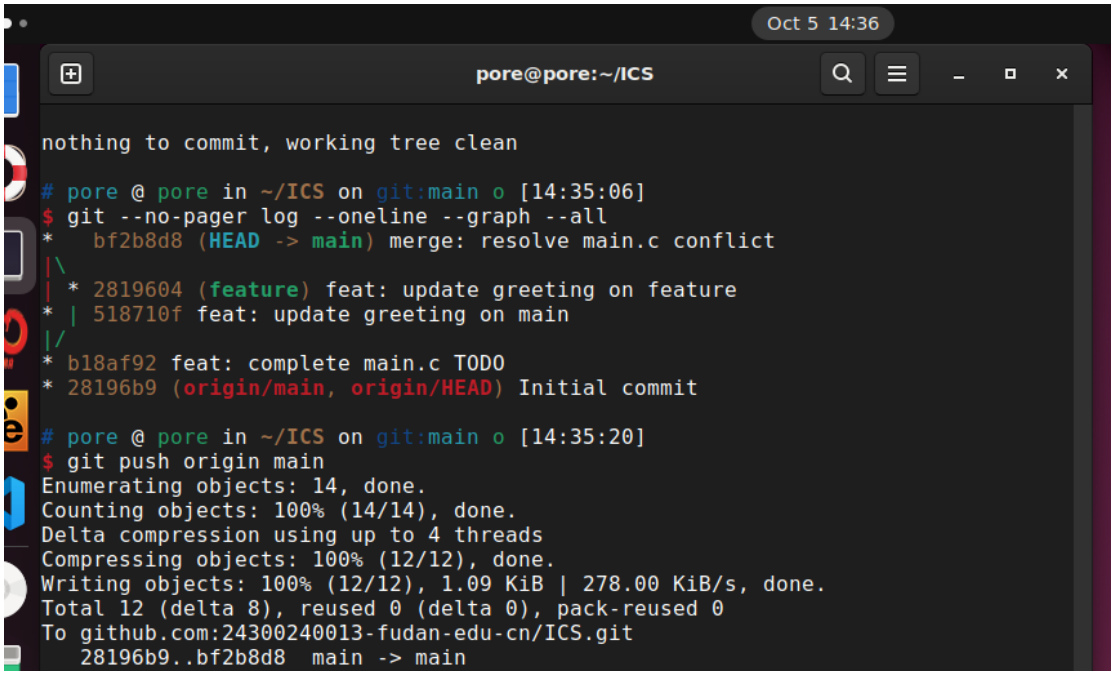
```
git status
git --no-pager log --oneline --graph --all
```

终端显示 working tree clean，提交图显示 main 和 feature 的开发历史已通过合并提交连接。

随后执行：

git push origin main

推送成功，远程 main 分支由 28196b9 更新为 bf2b8d8。



三、阅读概括及学习体会

1. Commit Message 规范

文章介绍了规范提交信息的作用，以及 Angular 提交信息规范。提交信息分为 Header、Body 和 Footer，其中 Header 必填，格式为：

<type>(<scope>): <subject>

type 表示修改类型，如 feat 表示新增功能、fix 表示修复问题、docs 表示文档修改；scope 表示影响范围，可以省略；subject 是简短描述。Body 用于详细说明修改，Footer 可用于说明不兼容变动或关闭相关 Issue。

统一的提交格式有助于快速理解历史、筛选提交以及自动生成 Change Log。本次实验中的“feat: complete main.c TODO”采用了类型与简短描述相结合的形式。

参考资料：
https://www.ruanyifeng.com/blog/2016/01/commit_message_change_log.html

2. 语义化版本

语义化版本以明确的公共 API 为基础，采用“主版本号.次版本号.修订号”的格式：

- 主版本号：发生不兼容的 API 修改。
- 次版本号：以向后兼容的方式新增功能。
- 修订号：以向后兼容的方式修复问题。

版本号还可以附加预发布标识和构建信息。已发布版本的内容不应直接修改，新的修改应作为新版本发布。

这一规范能够表达版本变化和兼容性，帮助使用者判断升级风险，减少依赖管理中的混乱。

参考资料：https://semver.org/lang/zh-CN/

3. 为什么要学习 Git？

Git 能够记录修改历史、比较版本，并恢复已经记录的内容，避免仅靠复制文件备份造成版本混乱。分支可以隔离不同开发任务，合并则用于整合成果。

结合我参与游戏开发的经历，Discord 主要解决沟通问题，而 Git 可以补充管理代码、配置和文档的版本。对于较大的音频等二进制资源，可以结合 Git LFS 等工具管理，但这类文件不能像文本代码一样直接合并内容。

通过本次实验，我理解了工作区、暂存区和仓库之间的关系，掌握了提交、分支和合并的基本操作。主动制造并解决冲突，也让我认识到冲突并不意味着代码丢失，而是需要开发者根据实际需求确定最终内容。